

Playing With the Android Database SQLite



Part-3

I'm back with another article in the evangelising series on the basics of developing apps for the Android platform. In this article, I will focus on building an Android app using an SQLite database. I assume readers are familiar with the basics of database operations and concepts. Here, we will also be using some threaded programming in Java, so I'm assuming readers are familiar with this concept.

The SQLite database is used in Android for storing and managing application data. The SQLite database is an open source lightweight transaction SQL database engine. More information about it can be found at <http://www.sqlite.org/>. There are other means of storing application data in Android rather than a database (they are known as Preferences), but those are more often used to store a very small amount of information, like usernames and passwords, so that users are not prompted for their credentials every time the application is started.

In this article, I will develop a simple log-in application, which stores user details in the database, and performs basic DDL (Data Definition Language, e.g., creation of tables, etc); DML (Data Manipulation Language, e.g., insert, delete, etc) transactions on the database. In this application, I have dealt with creating a user interface too: I have added a splash screen for the application, and navigation within the application is altered to enhance usability. Hence, this article also takes you to the next level of designing a user interface for your application.

Let's get started with the design of the application (see Figures 1 and 2).

In this application, we will create a database that holds the credentials of all the users. In this application, there are two types of user log-ins:

User login: Whenever someone tries to log in to the application, we query the credentials entered by that person against our database. If the entered credentials are correct, then we allow the person to log in, otherwise we display a suitable message.

Admin login: We also allow the admin to see the credentials of all the registered users. The admin user account is created during the installation of the application.

Now, let's get started with the implementation of the application. The completed Android application project will

look like what's shown in Figure 3.

We start with the creation of a splash screen for our application. Here, we will be using some threaded programming, as stated earlier.

```
setTheme(android.R.style.Theme_Translucent_NoTitleBar);
setContentView(R.layout.splashscreen);
```

```
try {
    int waited = 0;
    while(!_active && (waited < _splashTime)){
        sleep(100);
        if(!_active) {
            waited += 100;
        }
    }
} catch(InterruptedException e) {
    // do nothing
} finally {
    finish();
    startActivity(new Intent(SplashScreen.this,
        DatabaseActivity.class));
    stop();
}
```

The above snippet, from the "run" method, is the heart of a threaded program in Java, where all the work associated with a thread is done. The motive of this thread is to delay the functioning of the main thread for 5 seconds, and then return back to the main thread, which takes you to the log-in page of the application. Thus, we create a splash screen. We set the theme of this activity as one of the built-in themes—`android.R.style.Theme_Translucent_NoTitleBar`, and set the layout of the activity to our custom-made layout, which is "`R.layout.splashscreen`".